

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

PRACTICES FOR SECURE SOFTWARE REPORT....1
DOCUMENT REVISION HISTORY....3
CLIENT....3
INSTRUCTIONS....3
DEVELOPER....4
1. ALGORITHM CIPHER....4
2. CERTIFICATE GENERATION....8
3. DEPLOY CIPHER....9
4. SECURE COMMUNICATIONS....9
5. SECONDARY TESTING....11
6. FUNCTIONAL TESTING....13
7. SUMMARY....15
8. INDUSTRY STANDARD BEST PRACTICES....16
AI USAGE DISCLOSURE....17
REFERENCES....18

Document Revision History

Version	Date	Author	Comments
0.5	10/18/2026	Nathaniel Challender	First Draft
1.0	10/18/2026	Nathaniel Challender	Final Pass

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer

Nathaniel Challender

1. Algorithm Cipher

Artemis Financial wants to add file verification to its web application in the form of a checksum to confirm that data transferred through has not been altered. Stolen or altered data can result in financial loss, regulatory penalties, and damage to customer trust. As the developer and security expert on this project, my recommended algorithm for this verification step is Secure Hash Algorithm (SHA) 256. SHA-256 is approved by the National Institute of Standards and Technology (NIST), supported natively in Java, and fast enough to handle requests without slowing down the application. Older algorithms such as MD5 and SHA-1 are available through Java as well, but they were not considered because they are vulnerable to collision attacks. A hash function was chosen for this step because it produces a checksum that changes completely if even one bit of data changes, allowing Artemis Financial to confirm that data transferred through its web application has not been tampered with.

SHA-256

SHA-256 is part of the SHA-2 family of hash functions defined in Federal Information Processing Standard (FIPS) 180-4 (NIST, 2015). Java provides SHA-256 natively through the `MessageDigest` class using the standard algorithm name “SHA-256”. It processes input of any length in 512-bit blocks to produce a fixed 256-bit output called a “message digest”, which displays as a 64-character hexadecimal string. The exact same input will always produce the same hash output, and any change of input will produce a completely different digest. In practice, Artemis Financial’s application server will compute the SHA-256 checksum of the data,

then the receiving side will compute it again and compare both values. If they match, the data arrives unchanged. If they don't, it was corrupted or altered in transit and should be rejected.

Hash Functions and Bit Levels

Hash functions turn input of any length into a fixed-length output and cannot be reversed. NIST-approved hash functions are designed to work one-way and be collision-resistant, meaning that input should not be able to map to a pre-specified output and any two distinct inputs should not map to the same output (Moody et al., 2024). Security strength is measured in bits, with each added bit doubling the work an attacker needs to do. SHA-256 provides 256 bits of preimage resistance, which guards against forging an input that matches a given checksum. Since collision strength is half of the output size, SHA-256 provides 128 bits of collision resistance. A brute-force collision search would take roughly 2^128 operations, which is not possible with today's computing power.

Random Numbers, Symmetric vs. Non-Symmetric Keys

SHA-256 does not use a key or random numbers. The same input will always produce the same checksum, which is why verification is possible. Keys and randomness still matter for protecting the checksum as it travels between systems. Symmetric algorithms like AES use the same encryption key to both encrypt and decrypt data, which is faster and more efficient for processing large datasets. Asymmetric algorithms like RSA use a public and private key pair and serve to secure TLS connections that move data between systems and provide digital signatures. For this application, the Java Keytool generated an RSA key pair and self-signed certificate for the HTTPS connection. During the TLS handshake, the server uses the key pair to prove its identity while both sides agree on a shared symmetric session key to encrypt the connection. Session keys and initialization vectors should be unpredictable. Iron-Clad Java

recommends using `SecureRandom` with the algorithm named explicitly, calling `nextBytes()` so it seeds from the platform, and reseeding periodically (Manico & Detlefsen, 2015).

History and Current State of Encryption Algorithms

Cryptography has evolved throughout history from simple substitutions to algorithms designed to resist quantum computers. The first recorded techniques appeared around 4,000 years ago when Egyptian scribes used hieroglyphic symbol substitution in the tomb of Khnumhotep II (Sidhpurwala, 2023). Around 100 BCE, Julius Caesar used the “Caesar Cipher,” another substitution cipher that relied on shifting letters by an agreed number of places, to communicate with his generals on the battle front.

Encryption technology expanded during World War II as manual, electronic, and eventually digital forms of cryptography were developed for secure military communications. The US Marine Corps’ Navajo Code Talkers created and used a code built from their own language that was never cracked by Japan (Jevec, 2001). Germany used a commercially produced electromechanical cipher machine called ENIGMA which was eventually cracked by Alan Turing and his team at Bletchley Park, building on research shared by Polish cryptanalysts after Poland was overrun by Germany in 1939 (Central Intelligence Agency [CIA], 2015). Engineer Tommy Flowers, who had built codebreaking equipment for Turing, went on to build Colossus, “the world's first large-scale programmable electronic digital computer,” to break Germany’s Lorenz cipher (Copeland, 2026, para. 3).

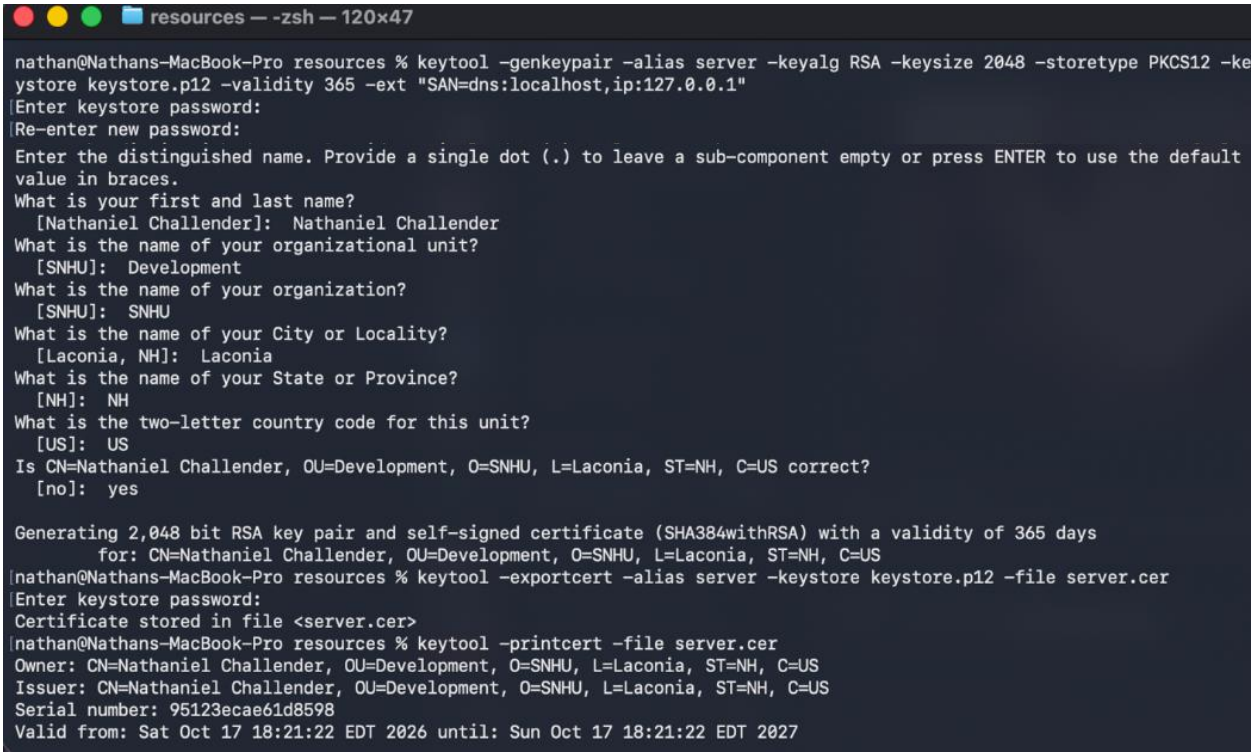
In the 1970s, DES became the United States’ federal encryption standard, and public-key cryptography emerged through Diffie-Hellman and RSA. In 1995, Netscape created SSL to secure internet communications, which became TLS in 1999. AES replaced DES in 2001, and GCM was standardized in 2007 (Dworkin, 2007).

Hash functions followed a similar path. Previous standards MD5 and SHA-1 were published in 1992 and 1995 respectively. Practical collision attacks broke MD5 in 2004 and

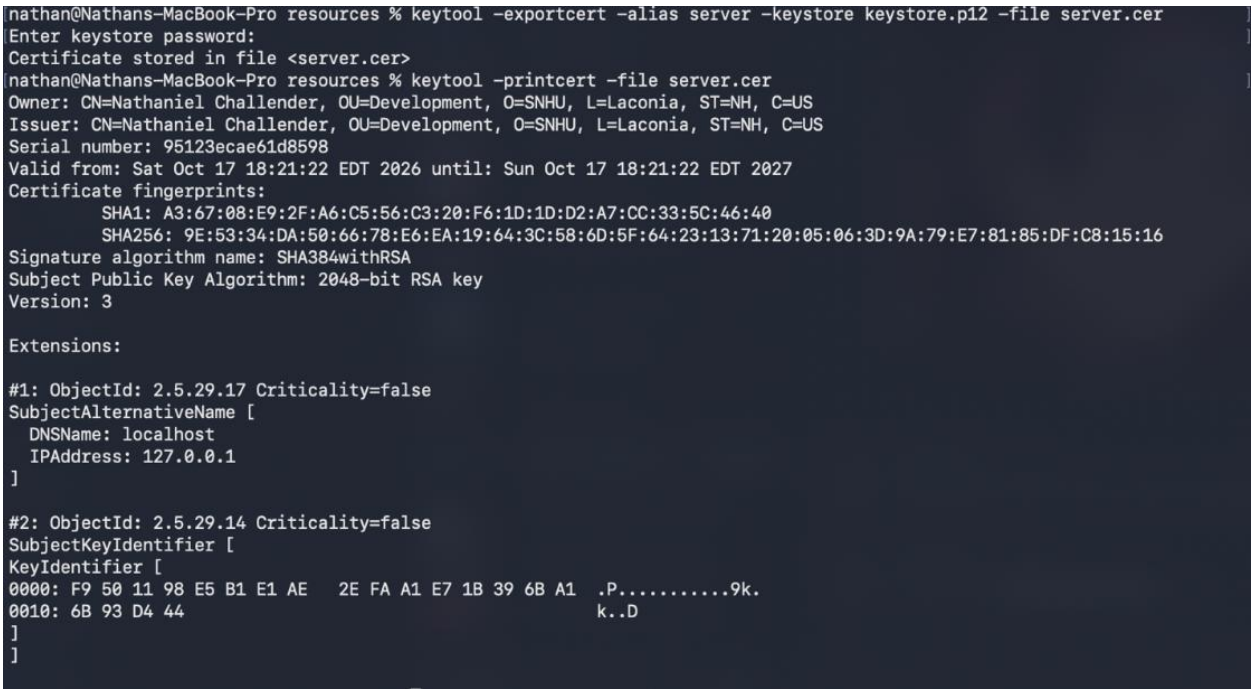
SHA-1 in 2017. NIST will be retiring SHA-1 completely in 2030 (NIST, 2022). More modern standards that have been published are the SHA-2 family, which includes SHA-256 and was published in 2002, and SHA-3 which was published in 2015.

Today, quantum computing is a looming threat for public-key algorithms like RSA and elliptic-curve cryptography. NIST released its first post-quantum standards in 2024 with a US target for completing the migration in 2035 (Moody et al., 2024). Quantum computing weakens but does not break hash functions like SHA-256, making SHA-256 less exposed and therefore a sound long-term choice for Artemis Financial's file verification.

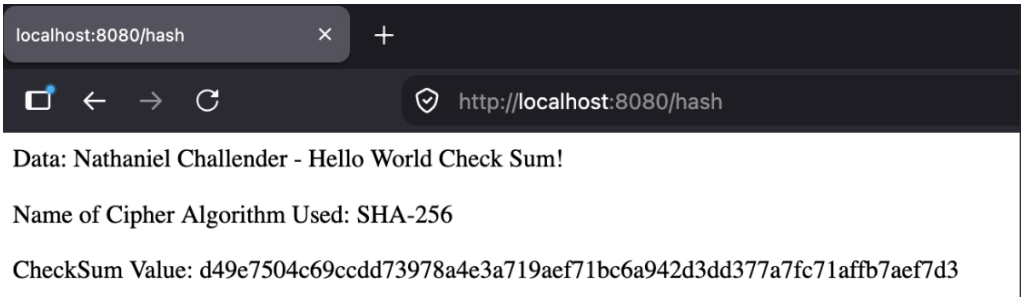
2. Certificate Generation



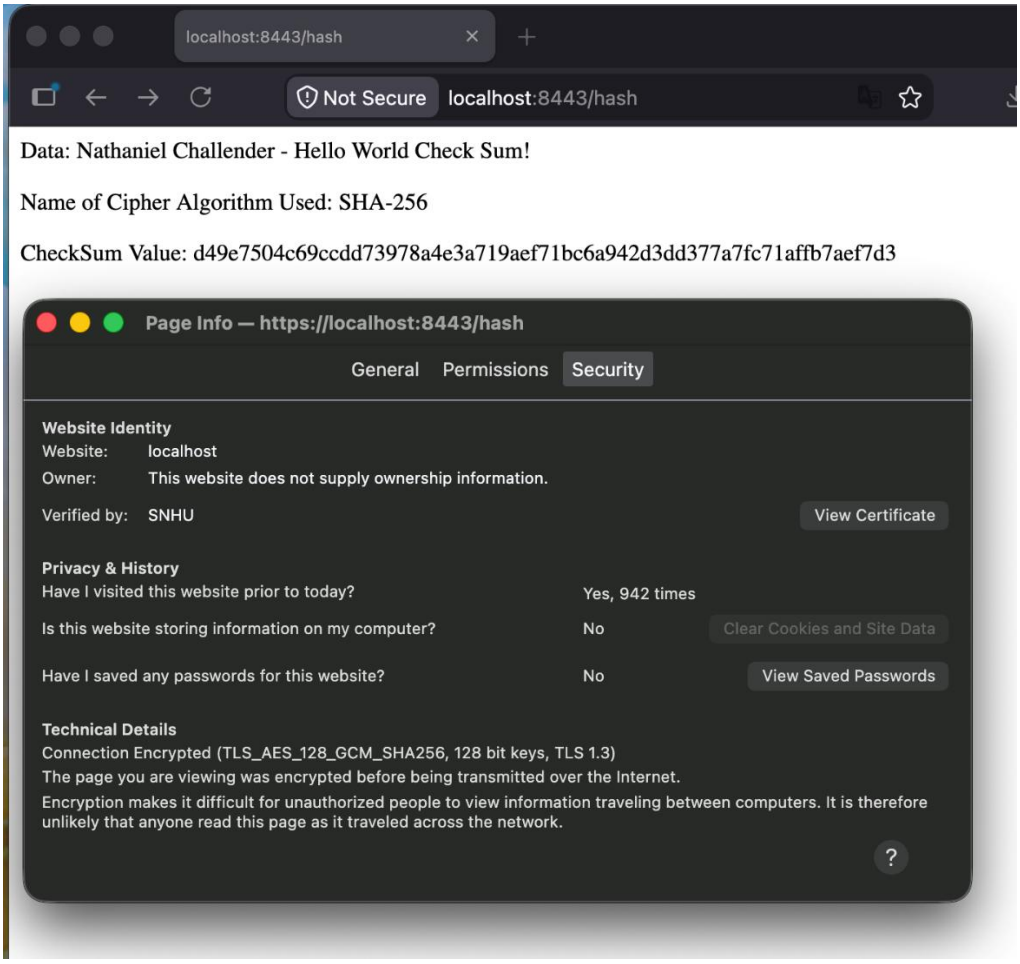
Note: The two images above were cropped to hide incorrect input which was overwritten in the second screenshot and did not add any value to the report.



3. Deploy Cipher



4. Secure Communications



Note: The certificate below was opened from the https://localhost:8443/hash page by clicking on the "Not Secure" label and following Connection not secure > More information > View Certificate. It is on the next page because Firefox opens it in a new tab.

Certificate

Nathaniel Challender

Subject Name

Country	US
State/Province	NH
Locality	Laconia
Organization	SNHU
Organizational Unit	Development
Common Name	Nathaniel Challender

Issuer Name

Country	US
State/Province	NH
Locality	Laconia
Organization	SNHU
Organizational Unit	Development
Common Name	Nathaniel Challender

Validity

Not Before	Sat, 17 Oct 2026 22:21:22 GMT
Not After	Sun, 17 Oct 2027 22:21:22 GMT

Subject Alt Names

DNS Name	localhost
IP Address	127.0.0.1

Public Key Info

Algorithm	RSA
Key Size	2048
Exponent	65537
Modulus	C3:1B:71:02:28:31:15:04:28:D3:99:2D:D8:CA:70:5C:17:4A:D4:40:18:55:FE...

Miscellaneous

Serial Number	00:95:12:3E:CA:E6:1D:85:98
Signature Algorithm	SHA-384 with RSA Encryption
Version	3
Download	PEM (cert) PEM (chain)

Fingerprints

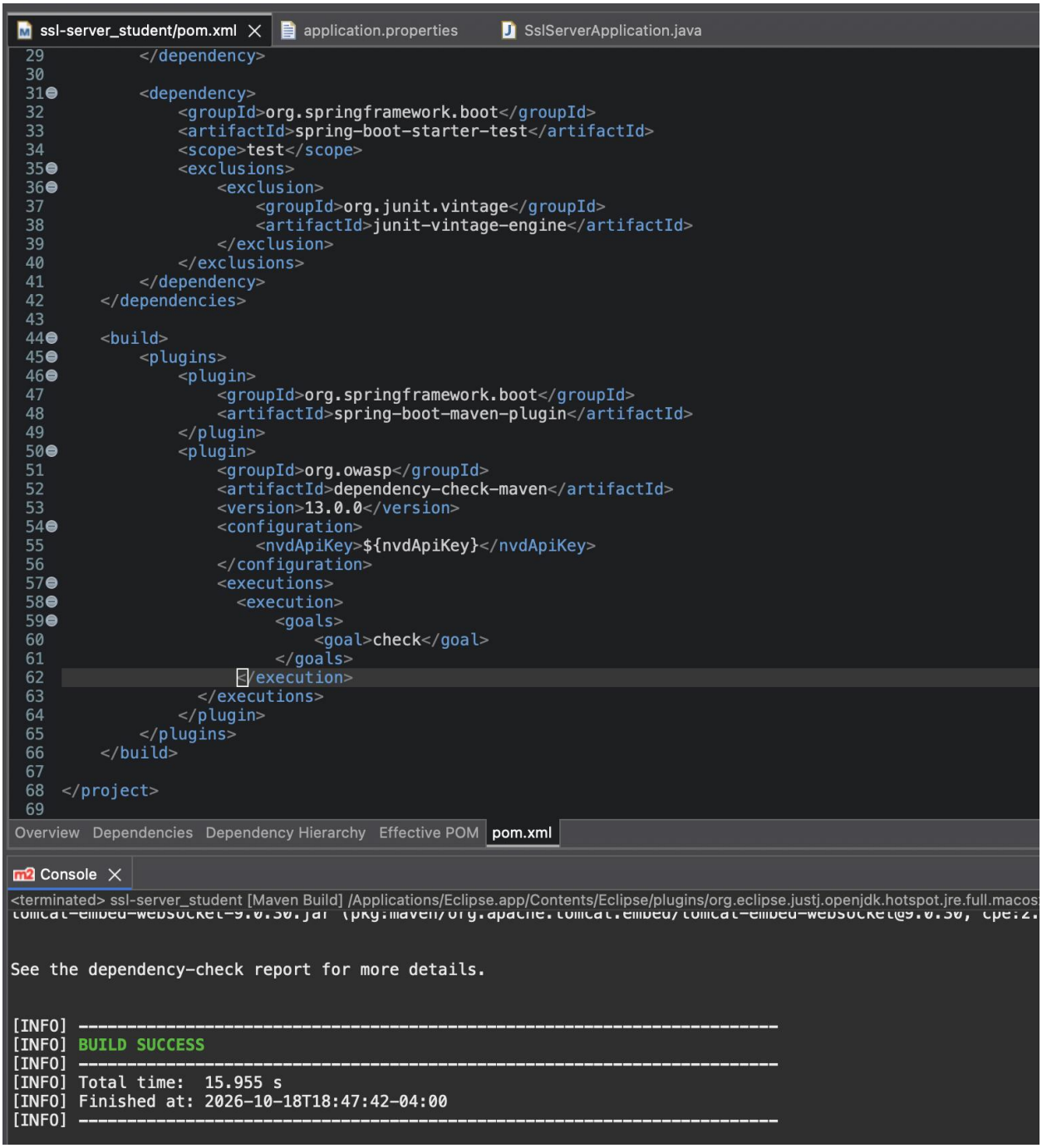
SHA-256	9E:53:34:DA:50:66:78:E6:EA:19:64:3C:58:6D:5F:64:23:13:71:20:05:06:3D...
SHA-1	A3:67:08:E9:2F:A6:C5:56:C3:20:F6:1D:1D:D2:A7:CC:33:5C:46:40

Subject Key ID

Key ID	F9:50:11:98:E5:B1:E1:AE:2E:FA:A1:E7:1B:39:6B:A1:6B:93:D4:44

5. Secondary Testing

No additional vulnerabilities were discovered after the code refactor. The refactored code relies on standard Java classes and classes included with Spring Boot and Tomcat. It does not add any new third-party libraries.



Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show less):

- *dependency-check version*: 13.0.0
- *Report Generated On*: Sun, 18 Oct 2026 18:47:42 -0400
- *Dependencies Scanned*: 49 (28 unique)
- *Vulnerable Dependencies*: 14
- *Vulnerabilities Found*: 254
- *Vulnerabilities Suppressed*: 0
- *NVD API Last Checked*: 2026-10-18T18:47:33-04
- *NVD API Last Modified*: 2026-10-18T22:16:30Z

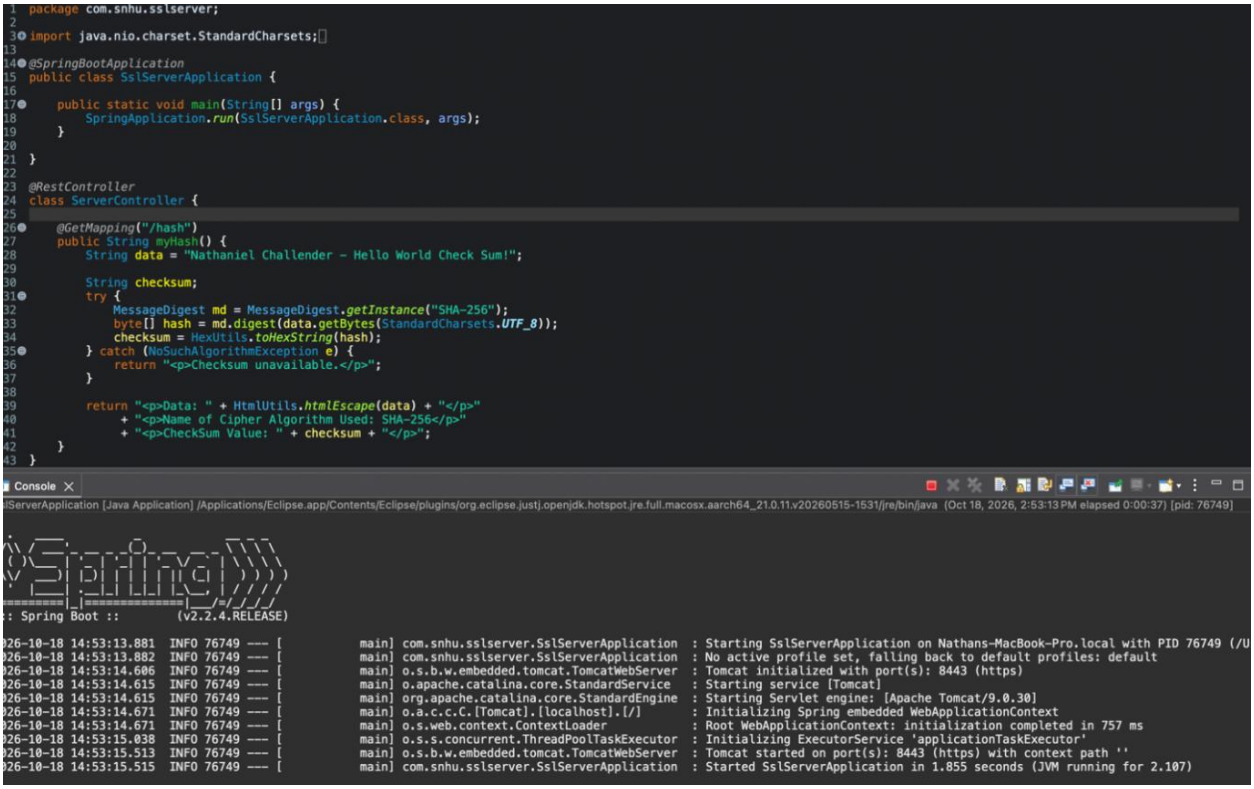
6. Functional Testing

After refactoring, I manually reviewed the code for errors in the syntax as well as logical or security flaws. I then ran the application to confirm there were no errors in the console. The code compiled without errors or warnings. Reloading the page and restarting the application produced the same checksum output every time.

During initial runs, I ran into a few bugs that caused the application to fail when the server did not start. I ran into a problem with where I stored the keystore password environment variable, which was set only in the Maven Build configuration and needed to be added to the run as Java Application configuration. I also misspelled the keystore type in `application.properties`. Once I corrected these errors, I restarted the application to verify they were resolved.

The checksum uses SHA-256 and, if the algorithm is not available, returns a generic message instead of a stack trace. The data string is HTML-encoded before it is displayed, preventing script injection attacks. The keystore password was generated with a secure password generator and is kept out of the source code. Due to the certificate being self-signed, the browser still shows a “Not Secure” warning, which can be resolved by replacing it with a certificate from a trusted certificate authority before the application is deployed to production.

The screenshot below shows the refactored code and console output showing a successful server start on port 8443 over HTTPS:



```
package com.snhu.sslserver;

import java.nio.charset.StandardCharsets;

@SpringBootApplication
public class SslServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SslServerApplication.class, args);
    }

}

@RestController
class ServerController {

    @GetMapping("/hash")
    public String myHash() {
        String data = "Nathaniel Challender ~ Hello World Check Sum!";

        String checksum;
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] hash = md.digest(data.getBytes(StandardCharsets.UTF_8));
            checksum = HexUtils.toHexString(hash);
        } catch (NoSuchAlgorithmException e) {
            return "<p>Checksum unavailable.</p>";
        }

        return "<p>Data: " + HtmlUtils.htmlEscape(data) + "</p>"
            + "<p>Name of Cipher Algorithm Used: SHA-256</p>"
            + "<p>CheckSum Value: " + checksum + "</p>";
    }
}
```

```
Console ×
lServerApplication [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.aarch64_21.0.11.v20260515-1531/jre/bin/java  (Oct 18, 2026, 2:53:13 PM elapsed 0:00:37) [pid: 76749]

.   ____          _            __ _ _
/\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
:: Spring Boot ::        (v2.2.4.RELEASE)

026-10-18 14:53:13.881  INFO 76749 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on Nathans-MacBook-Pro.local with PID 76749 (/U
026-10-18 14:53:13.882  INFO 76749 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to default profiles: default
026-10-18 14:53:14.606  INFO 76749 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443 (https)
026-10-18 14:53:14.615  INFO 76749 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
026-10-18 14:53:14.615  INFO 76749 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
026-10-18 14:53:14.671  INFO 76749 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
026-10-18 14:53:14.671  INFO 76749 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed in 757 ms
026-10-18 14:53:15.038  INFO 76749 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
026-10-18 14:53:15.513  INFO 76749 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https) with context path ''
026-10-18 14:53:15.515  INFO 76749 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 1.855 seconds (JVM running for 2.107)
```

7. Summary

The code for Artemis Financial's application was refactored to add a SHA-256 checksum route and enable HTTPS using a self-signed certificate. Four specific areas from the vulnerability assessment process flow diagram were addressed. Cryptography was addressed by using the SHA-256 algorithm through Java's `MessageDigest` class and a 2048-bit RSA key for the certificate. Client/server communication was made more secure by moving the application from the default HTTP on port 8080 to HTTPS on port 8443 with a self-signed certificate. Code error handling was addressed by adding a `NoSuchAlgorithmException` try/catch block to return a generic message explaining the error rather than exposing sensitive internal details through a stack trace. Code quality was addressed through the dependency-check scan and a manual review of the code line by line. The dependency-check scan flagged vulnerabilities with the project's existing libraries, which would likely require updating Spring Boot to the most recent release to resolve. Input Validation, APIs and Encapsulation were not addressed in this refactor because the /hash endpoint accepts no user input and handles no stored data. If future iterations of the application require hashing user-supplied files or authenticated access, they would need to be addressed.

Security was added in layers to cover gaps. Checksum detects altered data, HTTPS protects data and the checksum in transit, and the manual review and dependency-check scans verify secure coding practices and library-related vulnerabilities. After each change was made, I ran and tested the application again to find and correct errors until it ran without errors.

8. Industry Standard Best Practices

Established security standards and tools were used to create a secure application for Artemis Financial. Checksum uses SHA-256, which is a NIST approved algorithm that comes with the standard Java `MessageDigest` class. HTTPS is a secure extension of the HTTP protocol, incorporating a TLS/SSL layer to encrypt information in transit between client and server. The certificate's RSA key was randomly generated by the keytool. In production, keys should be rotated on a schedule, logged, and audited on a regular basis. Access to the keystore should be limited based on the principle of least privilege and keys should be stored in a key vault or other secure system separate from the application's source code. Error handling returns HTML-encoded generic messages and prevents information leaks through the console error log or stack trace. OWASP's Dependency-Check was used in the Maven Build to find known vulnerabilities in third-party libraries so they can be reviewed addressed.

Together, these practices support the well-being and security of Artemis Financial and their clients' data. As a financial institution, Artemis Financial is required to safeguard customer data under regulations such as the Gramm-Leach-Bliley Act (Federal Trade Commission, 2002). Industry standard and widely reviewed tools and algorithms are more stable and less likely to contain flaws compared to custom solutions. They are also easier to verify during audits. Using these tools lowers the risk of breaches that could cause financial and data loss, regulatory penalties, as well as damage to the company's image and loss of customer trust.

AI Usage Disclosure

Claude Sonnet 5.5 was used to format citations for the reference page and review final draft for grammatical and technical errors (Anthropic, 2026).

References

Anthropic. (2026) Claude Sonnet 5.5. [Large Language Model.] https://claude.ai/

Central Intelligence Agency. (2015). *The enigma of Alan Turing*. https://www.cia.gov/stories/story/the-enigma-of-alan-turing/

Copeland, B. J. (2026). The computer that helped win World War II. *IEEE Spectrum*. https://spectrum.ieee.org/colossus-computer-ieee-milestone

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf

Federal Trade Commission. (2002). *Standards for safeguarding customer information (16 C.F.R. Part 314).* https://www.ftc.gov/legal-library/browse/rules/safeguards-rule

Jevec, A. (2001). *Semper Fidelis, code talkers*. Prologue, 33(4). https://www.archives.gov/publications/prologue/2001/winter/navajo-code-talkers.html

Manico, J., & Detlefsen, A. (2015). *Iron-clad Java: Building secure web applications*. McGraw-Hill Education.

Moody, D., Perlner, R., Regenscheid, A., Robinson, A., & Cooper, D. (2024). *Transition to post-quantum cryptography standards (NIST IR 8547 ipd).* National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8547.ipd

National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (FIPS PUB 180-4). https://doi.org/10.6028/NIST.FIPS.180-4

National Institute of Standards and Technology. (2022, December 15). *NIST retires SHA-1 cryptographic algorithm*. https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm

Sidhpurwala, H. (2023). *A brief history of cryptography*. Red Hat Blog. https://www.redhat.com/en/blog/brief-history-cryptography